

ReTool

Reinforcement Learning for Strategic Tool Use in LLMs

Feng, Huang, et al. · ByteDance Seed

Where text-only reasoning hits a wall

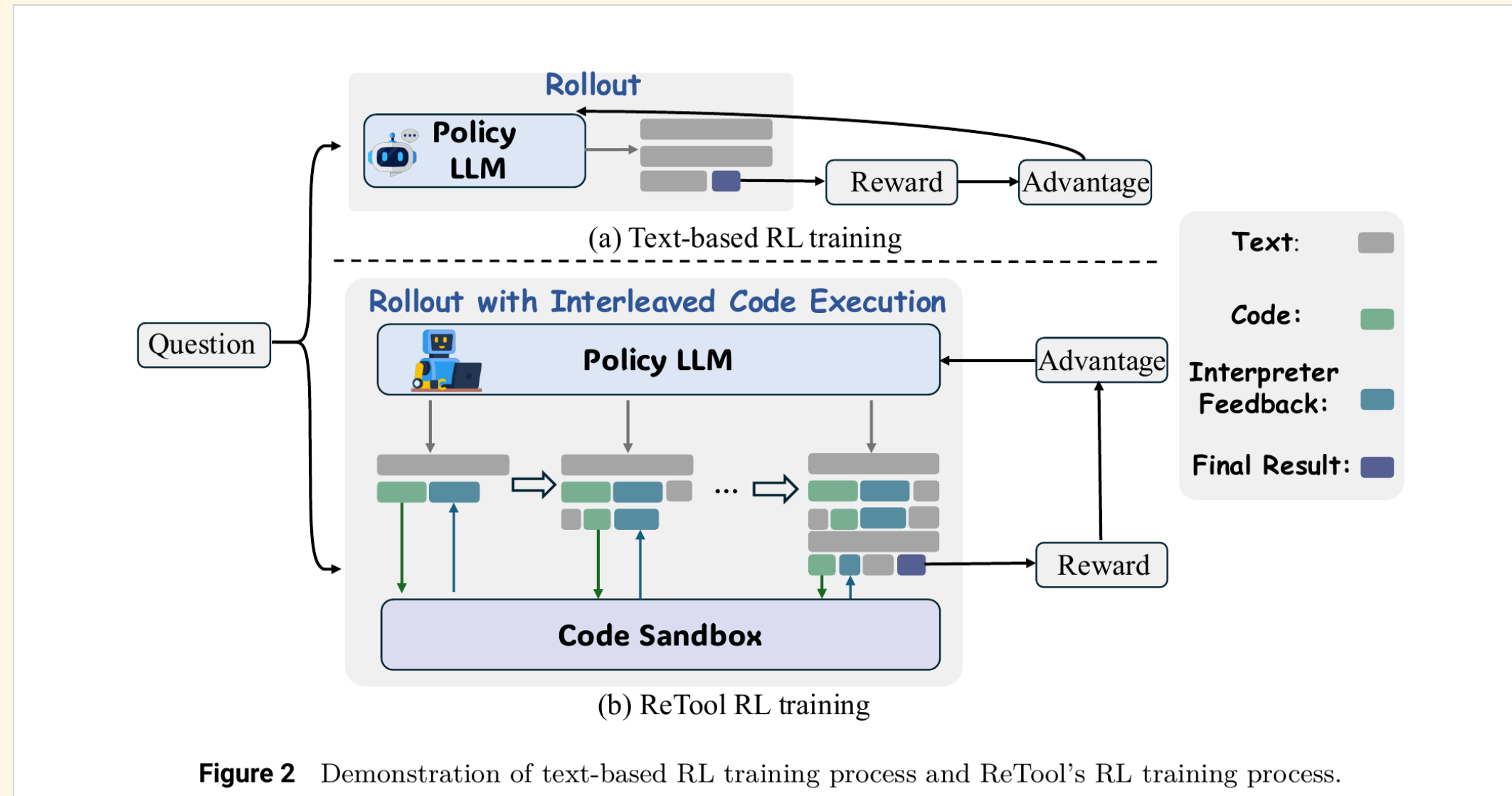
Reasoning models like OpenAI o1 and DeepSeek-R1 learn to produce long chains of thought purely in text, and this works very well for problems that reward patient, step-by-step argument.

But some problems are not naturally argument-shaped — they are naturally **computation-shaped**: geometric reasoning, exact arithmetic over many cases, solving a system of equations, checking 1,000 candidate values one by one. Doing this in pure text means the model has to carry out arithmetic and bookkeeping **inside its own token stream**, where a single slipped digit silently corrupts every step after it.

THE IDEA

A code interpreter does not make these mistakes. ReTool asks: instead of teaching a model to *talk about* running code, can reinforcement learning teach it to actually **stop, run real code, read the real output, and keep reasoning** — and to decide entirely on its own when that is worth doing?

From a text-only rollout to a rollout with real code execution



In ordinary text-based RL (top), the policy LLM generates one text rollout, which is scored once by a reward model, producing one advantage. In ReTool's RL (bottom), the rollout is built turn by turn: the policy writes a chunk of text and some code, a real **code sandbox** executes that code, and its output is spliced back into the same rollout before the model continues — repeating until a final answer is produced.

The rollout: one trajectory, three kinds of tokens

$$[t_1 \oplus c_1 \oplus f_1 \oplus \dots \oplus o]$$

t_1 — a chunk of natural-language reasoning the model writes; c_1 — a code block the model writes, marked off with `<code>...</code>` tags; f_1 — the sandbox's real feedback (printed output, or an error traceback if the code crashed), spliced back in between `<interpreter>...</interpreter>` tags; o — the final answer, inside `\boxed{}`; $\oplus$ — concatenation into one single token sequence.

Whenever the model emits `</code>`, generation pauses, the code is sent to a real sandbox, and the result is fed back before generation resumes — it can write more code, or give its final answer.

PROMPT GIVEN TO THE MODEL

“You now have the ability to selectively write executable Python code to enhance your reasoning process. The Python code will be executed by an external sandbox, and the output ... can be returned to aid your reasoning.” Nothing tells the model *when* to use this — that timing is exactly what RL has to teach it.

Before any RL: teaching the format with cold-start data

RL explores by trying variations of what a model already does — so the base model first needs *some* ability to use the `<code>` / `<interpreter>` protocol at all. ReTool builds this with a two-step pipeline, entirely before RL begins:

1. Collect existing, verified text-only reasoning traces (filtered by human review and by DeepSeek-R1 itself), giving a dataset D_{init} .
2. Automatically rewrite each trace: replace manual calculation steps with the equivalent code block and its real execution output, while leaving the surrounding reasoning — including any failed attempts — untouched. Keep only traces that pass both a format check and an answer-correctness check, giving a code-augmented dataset D_{CI} .

The base model is then supervised-fine-tuned on D_{CI} . This **cold start** is not where the model learns good tool-use strategy — it only learns the mechanics of the protocol. Strategy is what the RL stage has to discover.

Training objective: PPO over a hybrid trajectory

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}_{(q,a) \sim D, o_{\leq t} \sim \pi_{\theta_{old}}(\cdot | q)} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

$r_t(\theta) = \frac{\pi_{\theta}(o_t \mid q, o_{<t}; CI)}{\pi_{\theta_{old}}(o_t \mid q, o_{<t}; CI)}$ – ratio of the current policy's probability for token o_t to the old policy's; q, a – question and ground-truth answer, drawn from the training set D ; $o_{\leq t}$ – the rollout so far, a mix of reasoning, code, and interpreter-feedback tokens, sampled from the old policy $\pi_{\theta_{old}}$; the CI in $\pi_{\theta}(\cdot \mid q, o_{<t}; CI)$ marks that the real code interpreter's output is already sitting inside $o_{<t}$ when the model predicts o_t ; $\hat{A}_t$ – estimated advantage at token t ; ε – clip range.

NOT GRPO

Unlike DeepSeekMath and DeepSeek-R1, ReTool trains with plain **PPO**, which needs a learned value function to produce $\hat{A}_t$ – not GRPO's group-relative advantage from sampling many rollouts per question.

Intuition: the same clipped ratio, now over a longer story

The clipping mechanism itself is exactly the one from ordinary PPO/GRPO training: don't let a single update trust a rollout's advantage so much that it moves the policy's probability for a token far outside what the old policy actually assigned.

What's new is **what counts as the trajectory**. A token o_t here might be part of the model's own reasoning, part of a code block it chose to write, or part of the interpreter's feedback that got spliced in. The ratio $r_t(\theta)$ is computed the same way at every position — but credit for a good final answer now has to flow back through turns where the model *decided to hand control to the sandbox*, not just through turns where it kept talking.

WHY THIS MATTERS

This is exactly why the reward can't just check "did it reach the answer" in a vacuum — the policy is being credited for a strategy that includes *when* to call the tool, not only what it says.

Reward design: correctness only, nothing else

$$R(a, \hat{a}) = \begin{cases} 1, & \text{is_equivalent}(a, \hat{a}) \\ -1, & \text{otherwise} \end{cases}$$

a — the ground-truth answer; $\hat{a}$ — the model's final predicted answer, read out of its `\boxed{\}`; `is_equivalent` — a rule-based checker, not a learned model. That's the entire reward: $+1$ for a correct final answer, -1 for anything else.

INTUITION: NO CREDIT FOR MERELY RUNNING CODE

It would be tempting to also reward "code executed successfully" as a shaping signal. ReTool's authors deliberately leave this out: a reward for executability alone is something a model can collect by writing *any* code that runs, whether or not it helps — a direct invitation to reward hacking. A correctness-only reward forces every code call to earn its keep by actually contributing to the right final answer.

Two engineering details that make this trainable at scale

MASK THE INTERPRETER'S OWN TOKENS

The text between `<interpreter>...</interpreter>` did not come from the policy — it came from the sandbox. Those tokens are excluded from the loss entirely, so the model is never pushed to "predict" output it didn't generate, and its own coherent reasoning tokens aren't disrupted by foreign content sitting in the middle of them.

REUSE THE KV-CACHE ACROSS A CODE CALL

When `</code>` is detected, everything generated so far is cached. Only the new interpreter-feedback tokens need their KV-cache computed and appended when generation resumes — the whole prefix doesn't need to be recomputed, which is what makes multi-turn rollouts with real code execution affordable at RL scale.

Both are specific to the fact that part of every rollout now comes from outside the model — an asynchronous sandbox pool handles the actual code execution so slow executions don't stall the rest of training.

The headline result

Model	AIME2024 (pass@1)	AIME2025 (pass@1)
<i>Existing Baselines</i>		
Qwen2.5-Math-72B-Instruct	30.0	-
Qwen2.5-Math-72B-Instruct-TIR	40.0	-
Sky-T1	43.3	-
OpenAI o1-preview	44.6	37.9
DeepSeek-R1-Zero-Qwen-32B	47.0	-
QWQ-32B-Preview	50.0	33.5
s1-32B	56.7	-
<i>CI-powered RL</i>		
ReTool (Qwen2.5-32B-Instruct)	67.0	49.3
ReTool (DeepSeek-R1-Distill-Qwen-32B)	72.5	54.3
<i>Ablations on Qwen2.5-32B-Instruct</i>		
w/o Training (Base Model)	26.7	-
w/o CI (Text-based RL♠)	40.0	36.7
w/o RL (only Cold-start◇)	40.9	34.5

Table 1 Main results. ♠The Text-based RL method includes a text-based cold-start SFT initialization to ensure a fair comparison. ◇The inference process of the Cold-start model also incorporates code interpreter.

On Qwen2.5-32B-Instruct, ReTool reaches **67.0%** on AIME2024 and **49.3%** on AIME2025 — beating every listed baseline, including OpenAI's o1-preview (44.6% / 37.9%) and the 72B Qwen2.5-Math-Instruct-TIR model (40.0%). Swapping in a stronger backbone (DeepSeek-R1-Distill-Qwen-32B) pushes this further, to 72.5% / 54.3%.

Where do the gains actually come from?

CONFIGURATION	AIME2024	Δ VS. BASE
Base model, no training	26.7%	—
+ cold-start SFT only, no RL	40.9%	+14.2
+ RL, text-only, no code interpreter	40.0%	+13.3
+ RL, with code interpreter (ReTool)	67.0%	+40.3

THE SURPRISING COMPARISON

Cold-start SFT alone (40.9%) and RL without tool use (40.0%) land within a point of each other — **RL by itself, on pure text, buys almost nothing here** that SFT hadn't already bought. The 27-point jump only shows up once RL is combined with the ability to call a real code interpreter: the gain is specifically from learning *when and how* to use the tool, not from RL or tools alone.

Same destination, far fewer steps

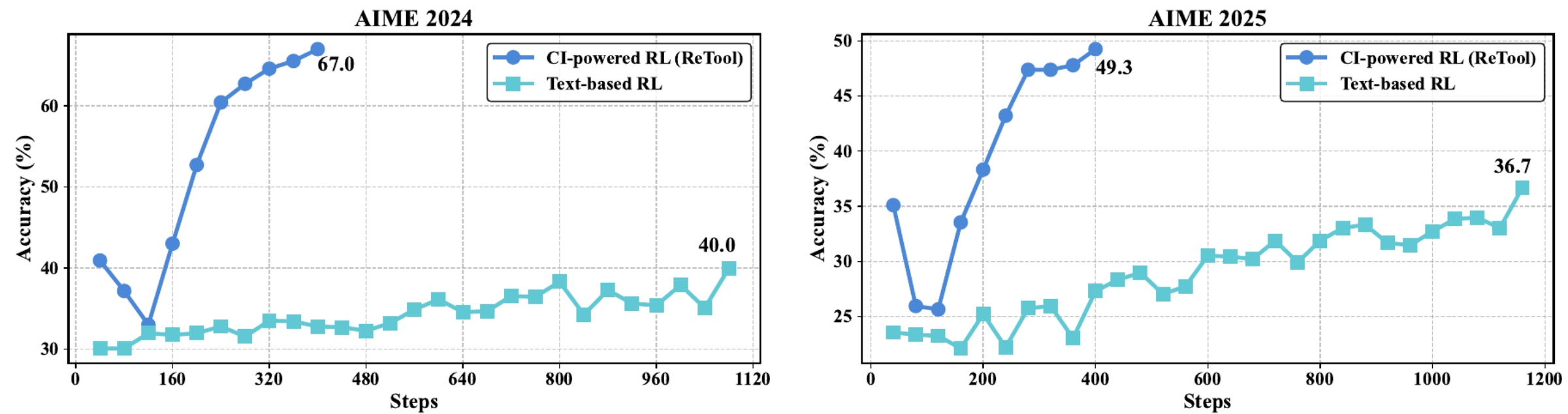


Figure 1 AIME 2024 & 2025 scores of ReTool and text-based RL baseline on the Qwen2.5-32B-Instruct model.

ReTool's accuracy curve (blue) shoots up almost immediately and plateaus around step 400. The text-based RL baseline (teal) is still climbing slowly past step 1000 and never catches up. ReTool reaches a *higher* accuracy using only about **37%** of the training steps the text-only baseline needs — tool use doesn't just raise the ceiling, it makes the exploration problem itself easier to solve.

Six signals of a strategy changing during RL

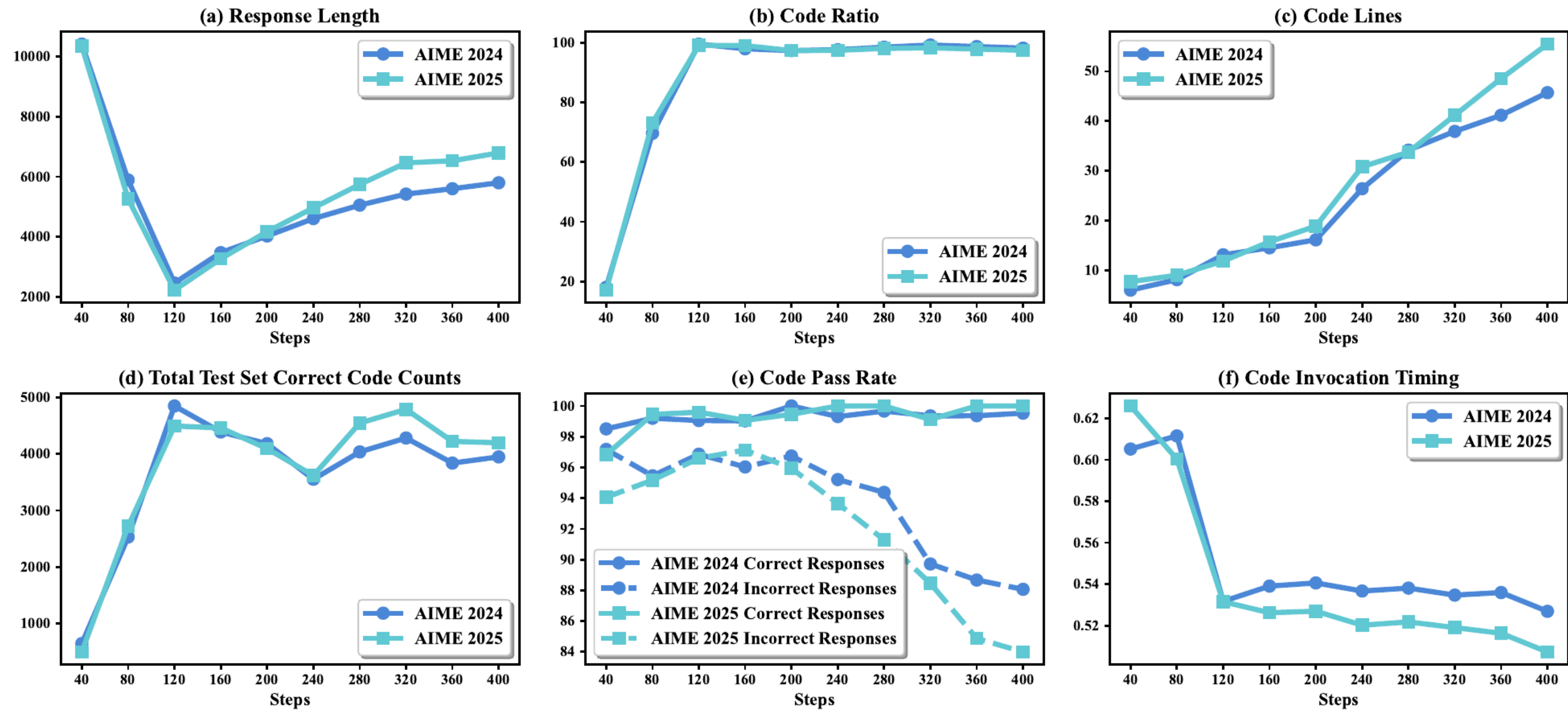


Figure 3 CI-related behavior evolution during RL training.

All six metrics are computed directly from the model's own generated responses at each saved RL checkpoint — nothing here was specified by hand; it all emerges from optimizing the single correctness reward.

Reading the six panels together

- **(a) Response length** falls sharply at first (code replaces laborious manual calculation), then rises again as the model learns more elaborate code strategies — ending about 40% shorter than before RL (10k → 6k tokens).
- **(b) Code ratio** climbs from under 20% to nearly 98% of responses: the model goes from rarely bothering with code to using it almost everywhere.
- **(c) Code lines** keep rising throughout training — nearly 5× longer by the end — as the model's code tackles more of the problem, not just the easy arithmetic.
- **(d)** correct code counts on the test set rise 5× alongside it.
- **(e) Code pass rate** stays near 100% for responses that end up correct, but *declines* for ones that end up wrong — buggy code is a leading indicator of a wrong final answer.
- **(f) Code invocation timing** shifts earlier in the response: the model learns to reach for the tool sooner rather than reasoning in text first and only checking at the end.

An emergent “aha moment”: code self-correction

Question: From an unlimited supply of 1-cent coins, 10-cent coins, and 25-cent coins, ... Find the number of values of $\$N$ between $\$1$ and $\$1000$ inclusive for which the greedy algorithm succeeds.

Response: Alright, let's tackle this problem step by step using code to ensure accuracy. The goal is to determine for each value of $\$(N)$ from 1 to 1000 whether the greedy algorithm produces the minimum number of coins. If it does, we count that $\$(N)$. ..

```
<code>
'''python
def test_specific_N(N_values):
    for N in N_values:
        greedy_result = greedy(N)
    ...
test_results = test_specific_N(N_values_to_test)
print(test_results)
'''
</code>
<interpreter>Traceback (most recent call last):
File "/tmp/tmpbr3uj7zz/tmpdmj39k6o.py", line 11, in <module> test_results = test_specific_N(N_values_to_test)
File "/tmp/tmpbr3uj7zz/tmpdmj39k6o.py", line 5, in test_specific_N greedy_result = greedy(N)
NameError: name 'greedy' is not defined
</interpreter>
```

Oops, the functions need to be defined in the same scope. Let's correct that:

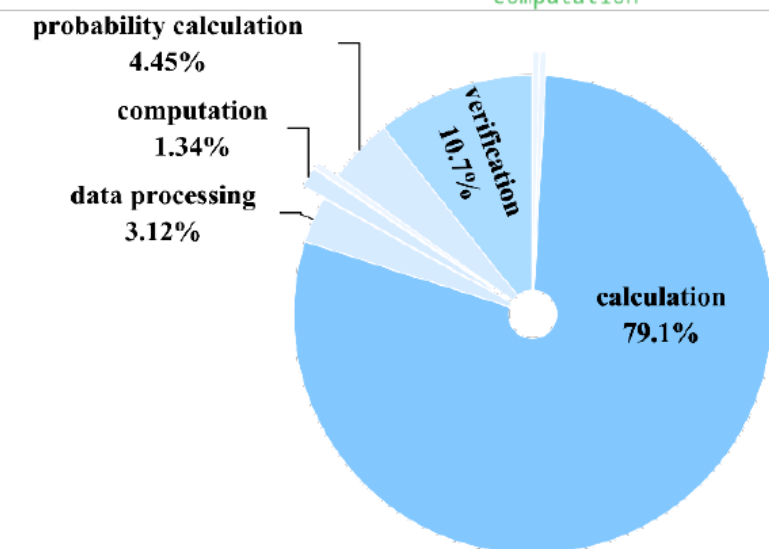
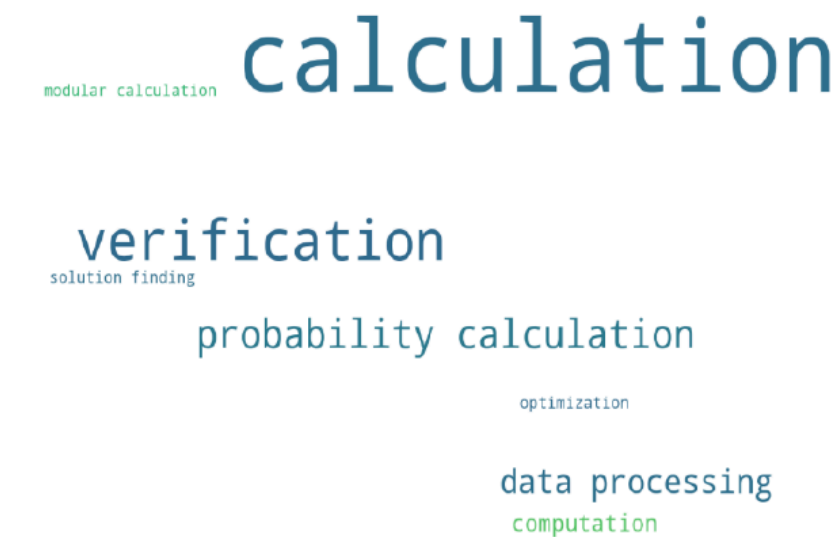
```
<code>
'''python
def greedy(N):
    denominations = [25, 10, 1]
    ...
def test_specific_N(N_values):
    for N in N_values:
        greedy_result = greedy(N)
    ...
test_results = test_specific_N(N_values_to_test)
print(test_results)
'''
</code>
<interpreter>{42: (9, 6), 25: (1, 1), 10: (1, 1), 1: (1, 1), 1000: (40, 40)}
</interpreter>
...
<answer> \boxed{610}
</answer>
```

Figure 4 The case of “aha moment” about code self-correction.

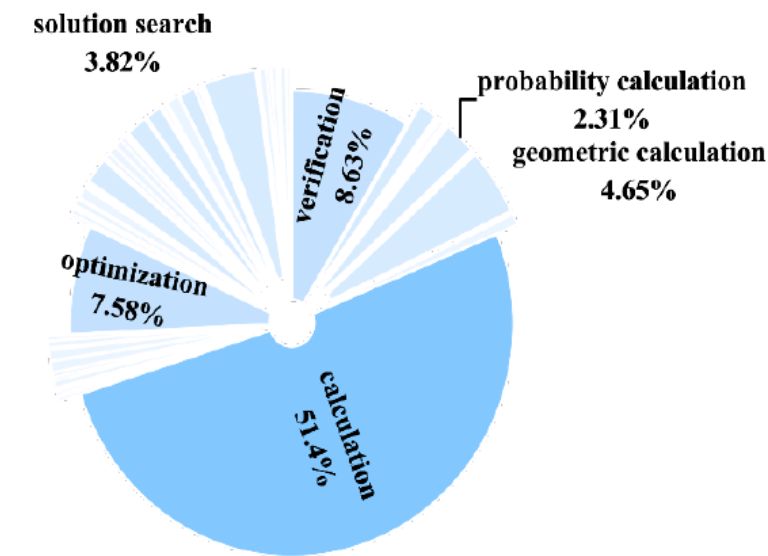
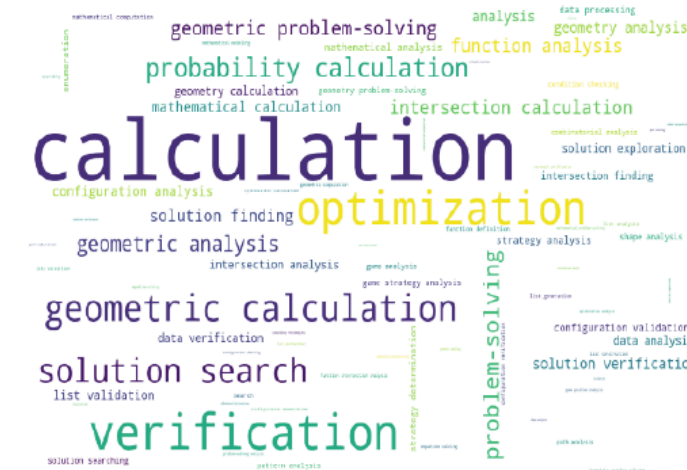
Here the model's first code block calls a function, `greedy()`, that was never defined — the sandbox returns a real `NameError` traceback. The model reads that error, responds “*Oops, the functions need to be defined in the same scope. Let’s correct that*”, and rewrites working code.

No training example ever explicitly demonstrated this fix-after-error behavior — it emerged purely because the correctness reward only pays off if the model keeps going after a crash instead of giving up.

What the code is actually for shifts, too



(a) Before Retool RL training.



(b) After Retool RL training.

Figure 5 Code purpose analysis.

Before RL, code is used almost entirely for raw **calculation** (79.1% of code snippets). After RL, calculation's share drops to 51.4%, and new categories gain ground: optimization, geometric calculation, solution search. The model isn't just calculating more accurately — it's discovered more distinct **roles** code can play in a solution.

Same question, two reasoning styles

Question: Let N be the greatest four-digit positive integer with the property that whenever one of its digits is changed to 1, the resulting number is divisible by 7. Let Q and R be the quotient and remainder, respectively, when N is divided by 1000. Find $Q+R$.	
Text-based Reasoning Before RL: Okay, so I need to find the greatest four-digit number N such that if any one of its digits is replaced by 1, the resulting number is divisible by 7. Then, I have to find Q and R when N is divided by 1000, and then compute $Q + R$. Alright, let's break this down step by step... Therefore, each of these equations can be rearranged to: 1. $N \equiv -(1 - A) \cdot 1000 \pmod{7}$ 2. $N \equiv -(1 - B) \cdot 100 \pmod{7}$ 3. $N \equiv -(1 - C) \cdot 10 \pmod{7}$ 4. $N \equiv -(1 - D) \cdot 1 \pmod{7}$ Therefore, all four expressions must be equal to $N \pmod{7}$. Therefore, we can set them equal to each other: $-(1 - A) \cdot 1000 \equiv -(1 - B) \cdot 100 \equiv -(1 - C) \cdot 10 \equiv -(1 - D) \cdot 1 \pmod{7}$ Wait, let's check B, C, D for $A=9$: $B=3 \cdot 1000 \pmod{7} = 3 \cdot 9 \pmod{7} = 27 \pmod{7} = 6$ $C=2 \cdot 100 \pmod{7} = 2 \cdot 9 \pmod{7} = 18 \pmod{7} = 4$ $D=6 \cdot 10 \pmod{7} = 6 \cdot 9 \pmod{7} = 54 \pmod{7} = 5$ So, $B=4, C=3, D=0$. Therefore, $N=9430$... Let me compute each of these modulo 7: 1. $1000 \equiv 6 \pmod{7}$, so $6 + 100 \cdot B + 10 \cdot C + D \equiv 0 \pmod{7}$ But $100 \cdot B \equiv 2 \cdot B \pmod{7}$, $10 \cdot C \equiv 3 \cdot C \pmod{7}$, so: $6 + 2B + 3C + D \equiv 0 \pmod{7}$ 2. $1000 \cdot A \equiv 6A \pmod{7}$, $100 \equiv 2 \pmod{7}$, so: $6A + 2 + 3C + D \equiv 0 \pmod{7}$ 3. $1000 \cdot A + 2B + 3 + D \equiv 0 \pmod{7}$ (since $100 \cdot B \equiv 2B$, $10 \cdot 1 \equiv 3$, $10 \cdot C \equiv 3C$) Wait, no. Let me recompute each equation properly. Equation 1: $1BCD \equiv 0 \pmod{7}$ Which is $1000 \cdot 1 + 100 \cdot B + 10 \cdot C + D \equiv 0 \pmod{7}$ $1000 \equiv 6, 100 \equiv 2, 10 \equiv 3, \dots$ Let me tabulate A from 1 to 9: $A \mid B=5-4A \pmod{7} \mid C=6-5A \pmod{7} \mid D=2-A \pmod{7} \mid \text{Valid?}$ $1 \mid 5-4=1 \pmod{7} \mid 6-5=1 \pmod{7} \mid 2-1=1 \mid \text{Yes} \dots$ Result: - $N = 5624$ - Dividing by 1000 gives $Q = 5$ and $R = 624$ - $Q + R = 5 + 624 = 629$ $\boxed{629}$	CI-powered Reasoning After RL: Alright, let's tackle this problem step by step. The goal is to find the greatest four-digit positive integer N such that changing any one of its digits to 1 results in a number divisible by 7. Then, we need to find $Q + R$ where Q and R are the quotient and remainder when N is divided by 1000... First, let's compute the values of $1000 \pmod{7}, 100 \pmod{7}, 10 \pmod{7}$, and $1 \pmod{7}$: <pre><code> python # Compute modular values mod_1000 = 1000 % 7 mod_100 = 100 % 7 mod_10 = 10 % 7 mod_1 = 1 % 7 ... print(f"1 mod 7 = {mod_1}") </code></pre> $1000 \pmod{7} = 6, 100 \pmod{7} = 2, 10 \pmod{7} = 3, 1 \pmod{7} = 1$... We need to solve these equations for digits a, b, c, d where a is from 1 to 9 and b, c, d are from 0 to 9. Let's use code to iterate through possible values and find the valid N. <pre><code> python def find_valid_n(): for a in range(9, 0, -1): ... return N ... N = find_valid_n() Q = N // 1000 R = N % 1000 result = Q + R print(f"N = {N}, Q = {Q}, R = {R}, Q + R = {result}") </code></pre> $N = 5694, Q = 5, R = 694, Q + R = 699$ Thus, the final answer is confirmed to be 699. $\boxed{699}$

Figure 6 Case of CI-powered Reasoning vs. Text-based Reasoning.

Same four-digit-integer question, two transcripts. Text-only reasoning (left) hand-derives modular arithmetic across many lines and lands on the wrong answer, 629 — a single slipped step upstream corrupts everything after it. CI-powered reasoning (right) offloads the same arithmetic to two short code blocks and reaches the correct answer, 699, confirmed by the interpreter itself.

Tying back to the course's RL arc

- **DeepSeekMath** introduced GRPO: group-relative advantages from many sampled rollouts, no value function.
- **DeepSeek-R1** reused GRPO unchanged, but applied it to long, free-form reasoning chains verified only by a rule-based correctness reward (RLVR).
- **ReTool** keeps the same ingredients — a clipped policy-gradient objective, a simple rule-based outcome reward — but widens what a “rollout” is allowed to contain: not just the model's own tokens, but real feedback from an external tool the model chooses to call.

THE THROUGHLINE

Every one of these papers is the same underlying idea — sample trajectories, score them by whether they reached the right answer, push up the ones that worked — applied to a progressively richer notion of what a trajectory is: a single answer, then a long reasoning chain, then a reasoning chain that can reach out into the world and act. That progression is exactly what “agentic RL” means.

References

- Feng, Huang, Qu, Zhang, Qin, Zhong, Jiang, Chi, Zhong. *ReTool: Reinforcement Learning for Strategic Tool Use in LLMs*. ByteDance Seed, 2025. arxiv.org/abs/2504.11536
- Project page: retool-rl.github.io
- MarkTechPost. *ReTool: A Tool-Augmented Reinforcement Learning Framework for Optimizing LLM Reasoning with Computational Tools*. marktechpost.com
- Schulman, Wolski, Dhariwal, Radford, Klimov. *Proximal Policy Optimization Algorithms*. 2017. arxiv.org/abs/1707.06347 — the PPO algorithm ReTool trains with.
- DeepSeek-AI et al. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. 2025. arxiv.org/abs/2501.12948